Project	EventForce Management System (Salesforce)
Document	Development Phase 2: Backend Development and Configurations
Team ID	SWTID-2026-1405 | SmartBridge x SkillWallet
Team	Monika P (Team Leader), Usha S, Shobana M, Mujahidha Begam M
College	P.B. College of Engineering, B.Tech Information Technology

Development Phase 2: Backend Development and Configurations

This phase created the data model and all business logic of EventForce: objects, fields, relationships, validation, email templates, the approval process, the flow and the Apex code.

1. Custom objects and tabs

Six custom objects were created from Setup, Object Manager. Reports, activities, field history tracking (on Event) and search were enabled so that the objects work in the app, in reports and in global search. A custom tab was created for each object so users can open them from the Lightning app.

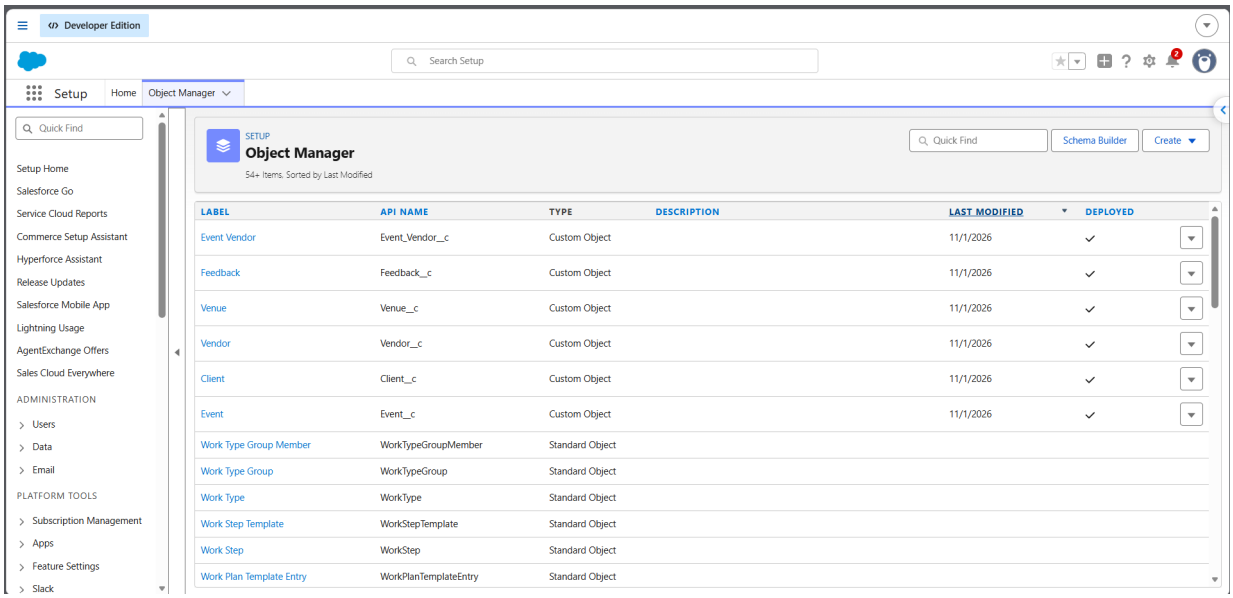


Figure 1: Custom objects in Object Manager

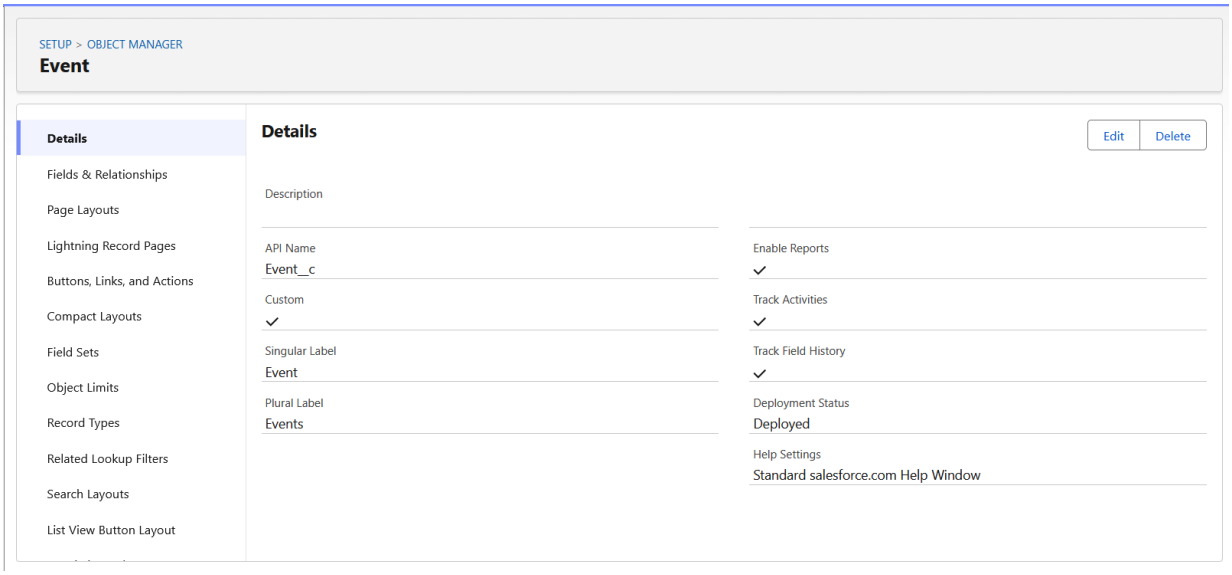


Figure 2: Event object definition (reports, activities and history tracking enabled)

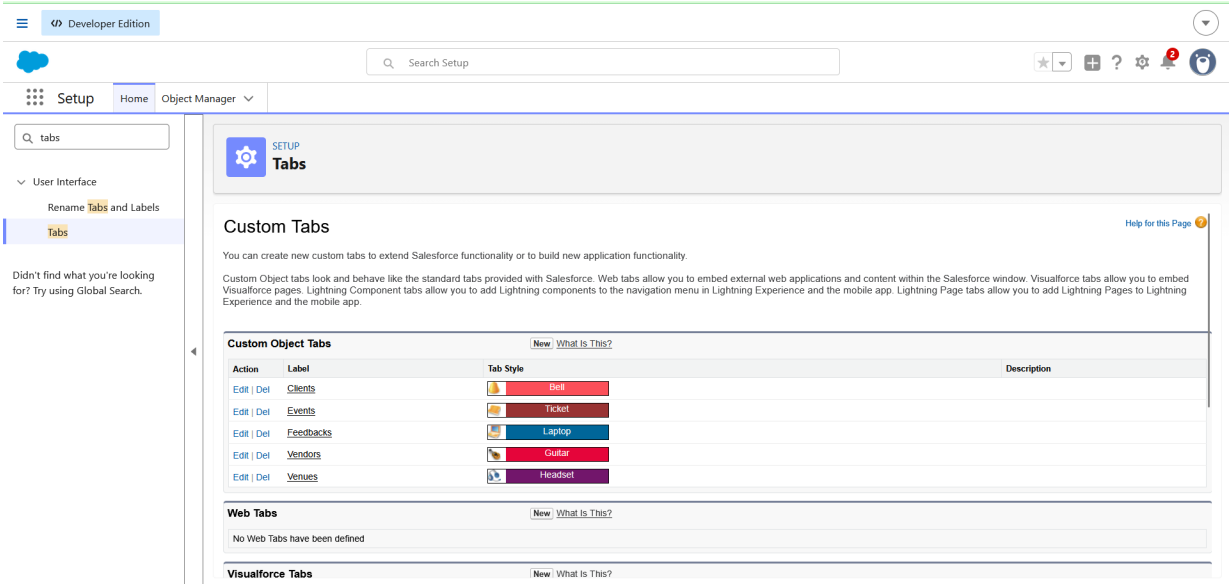


Figure 3: Custom tabs for Clients, Events, Feedbacks, Vendors and Venues

2. Fields

Event

Field	Type	Details
Event Name	Text (80)	Standard name field
Event Type	Picklist	Wedding, Corporate, Birthday, Anniversary, Festival, Concert, Other
Event Date	Date	Date of the event
Event Status	Picklist	Planned, Confirmed, Completed, Pending Cancellation, Canceled, Rejected
Event Budget	Formula (Currency)	Default budget from the event type
Venue	Lookup (Venue)	Where the event is held
Client	Lookup (Client)	Who booked the event

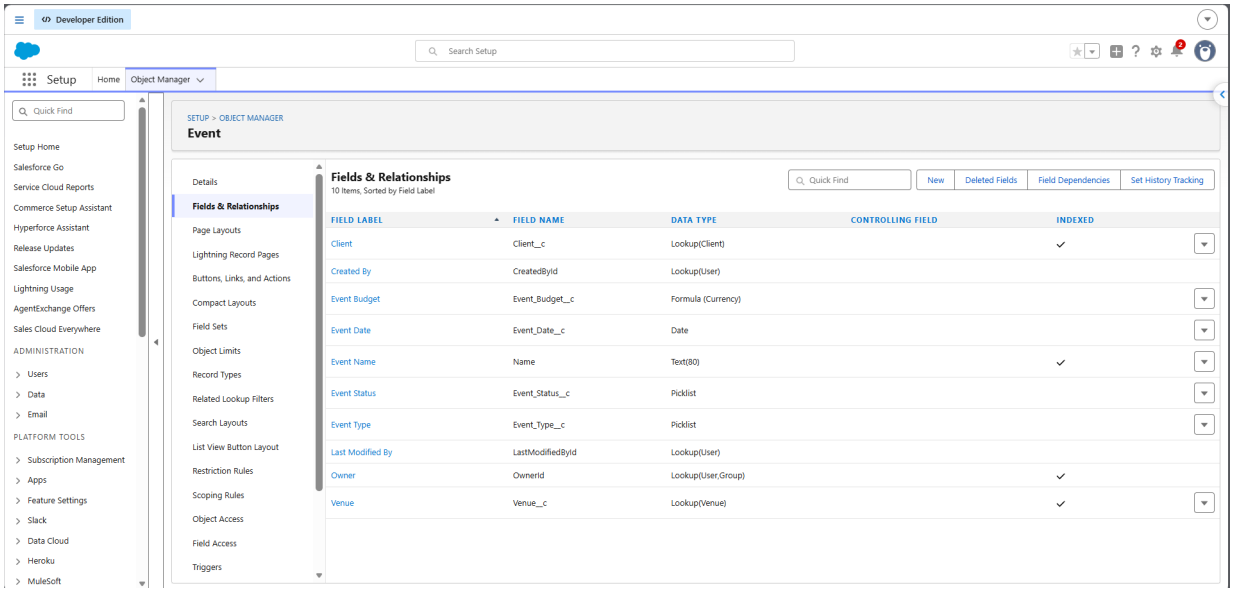


Figure 4: Event fields and relationships

Client, Venue, Vendor and Feedback

Object	Fields
Client	Email (Email), Phone (Phone), Address (Text Area), Country (Picklist), City (Picklist)
Venue	Address (Text Area), Location (URL), Capacity (Number), Availability Status (Available, Reserved, Not Available)
Vendor	Service Type (Multi-select: Catering, Decor, Photography, Videography, Lighting, Stage Setup, Makeup Artist, DJ/Music, Transportation, Hosting/Anchor), Email, Phone, Status (Available, Booked, Cancelled)
Feedback	Rating (1 to 5), Comments (Long Text Area), Event (Lookup), Client (Lookup). Record name is an auto number F-{0000}

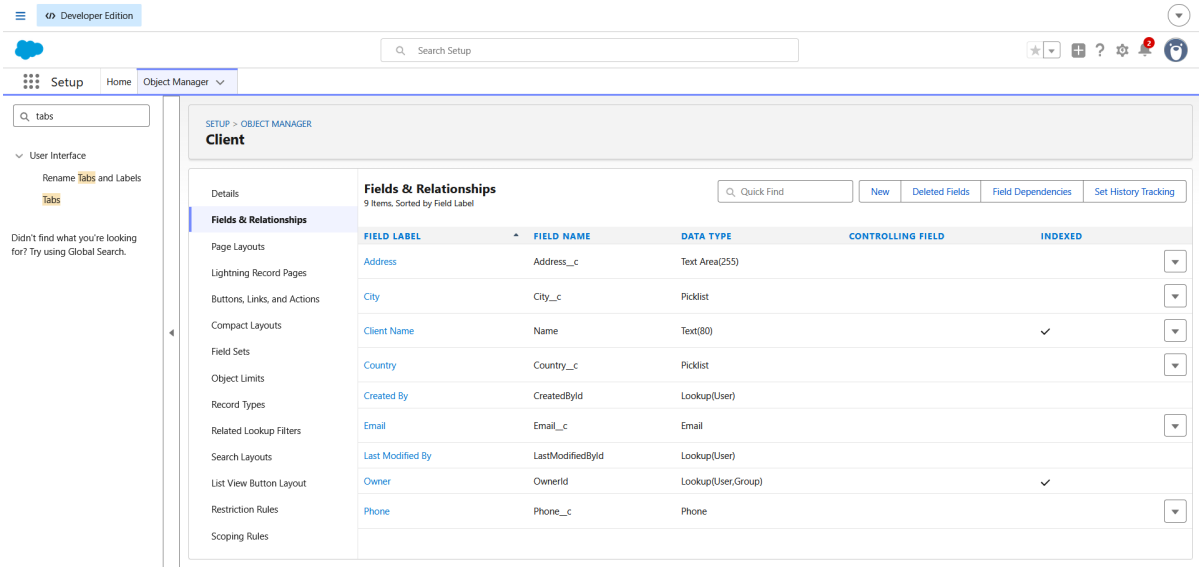


Figure 5: Client fields

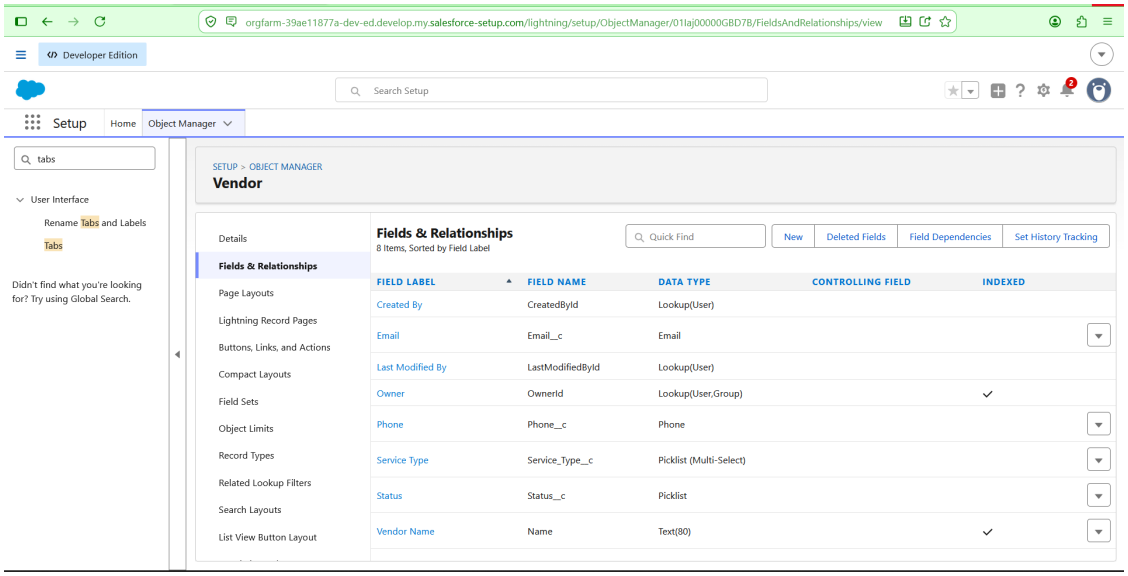


Figure 6: Vendor fields

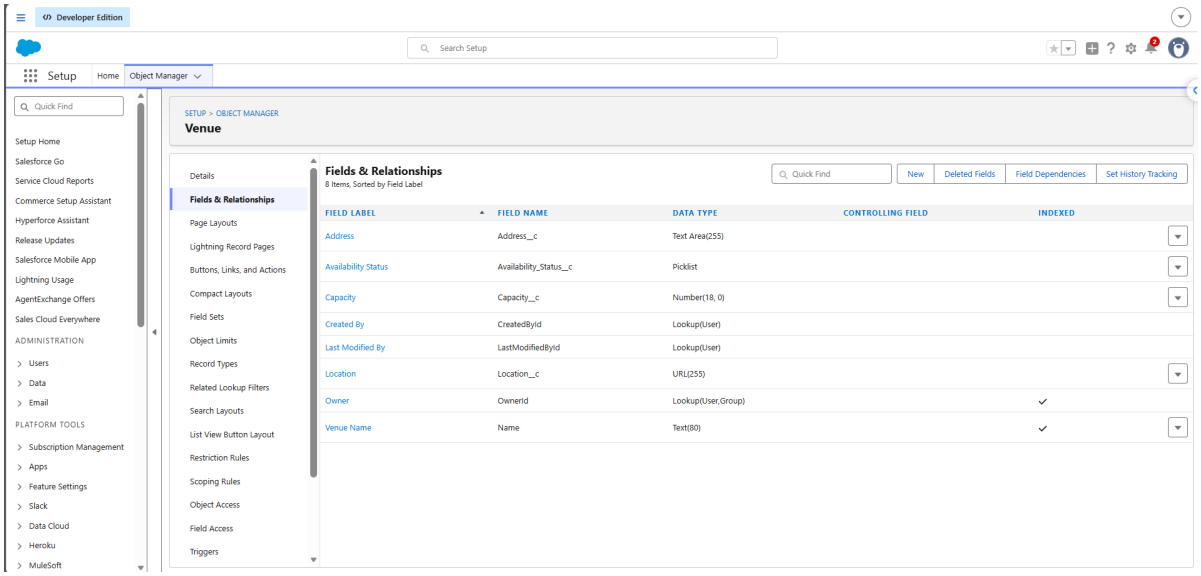


Figure 7: Venue fields

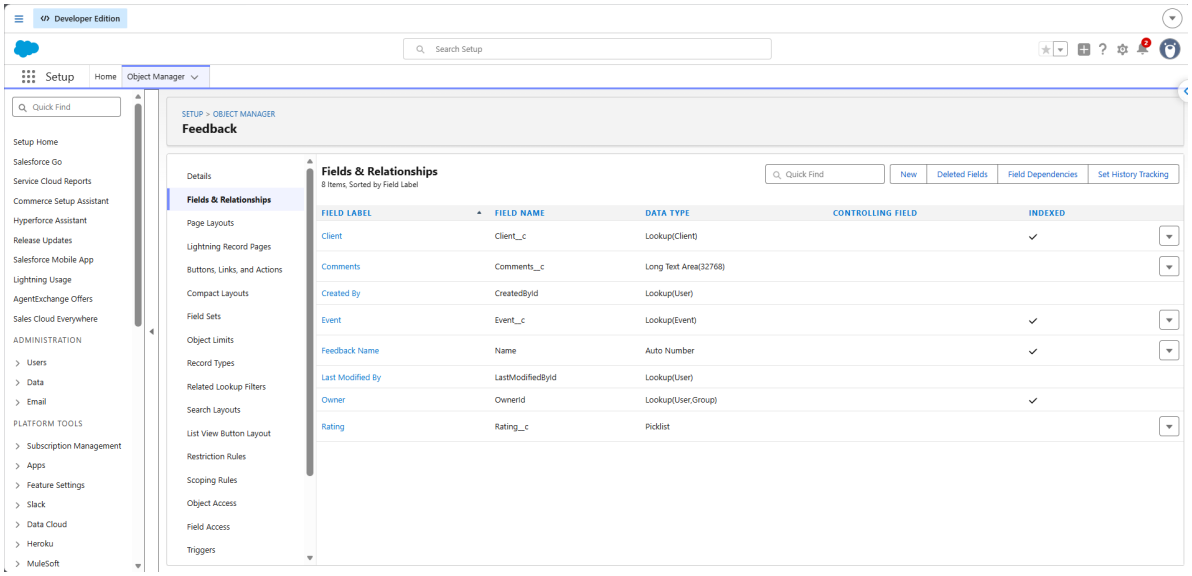


Figure 8: Feedback fields with Event and Client lookups

3. Relationships

Event Vendor is a junction object. It has two master-detail relationships, one to Event and one to Vendor, which creates the many-to-many link between events and vendors. On Feedback, a lookup filter on the Event field only offers events whose client is the same as the client chosen on the feedback record.

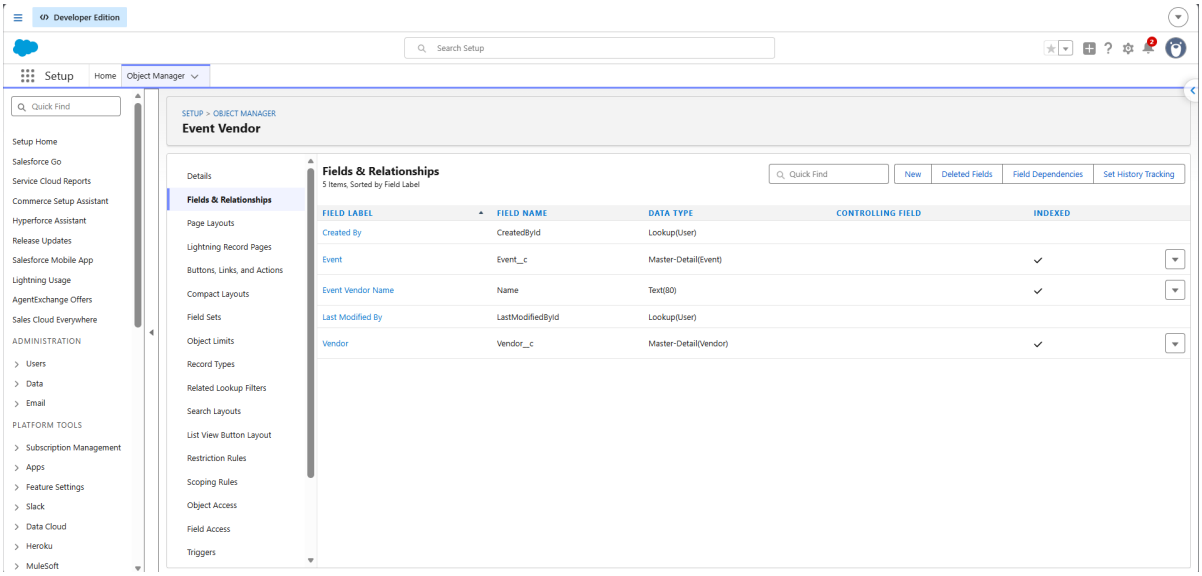


Figure 9: Event Vendor junction object with two master-detail fields

4. Formula field

Event Budget gives a default budget based on the event type.

```
CASE(TEXT(Event_Type__c),
  "Wedding", 50000, "Corporate", 30000, "Birthday", 10000, "Anniversary", 20000,
  "Festival", 60000, "Concert", 40000, "Other", 15000, 0)
```

5. Validation rule

The rule Email_Valid_Address on Client shows the message Please Enter Valid Email Address on the Email field when the address does not match the expected pattern.

```
NOT(REGEX(Email__c, "^[a-zA-Z0-9._]+@[a-zA-Z0-9.]+\\.[a-zA-Z]{2,}$"))
```

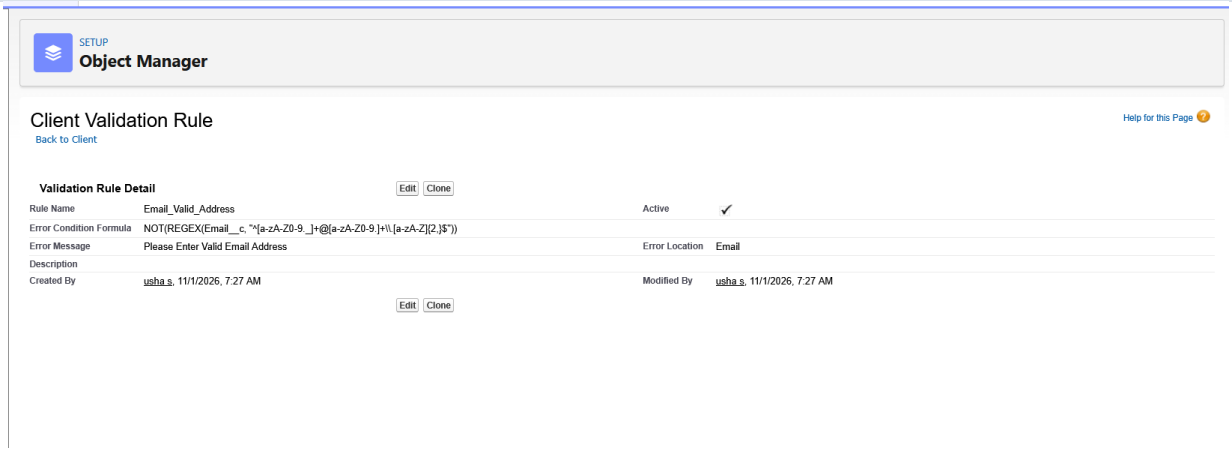


Figure 10: Validation rule on Client

6. Email templates

Four text email templates support the approval process: Event Cancellation Request Notification (sent to the Event Coordinator role on submission), Email to Client After Cancellation, Approval Notification to Event Owner and Rejection Notification to Event Owner. They use merge fields such as the event name, event date and owner name.

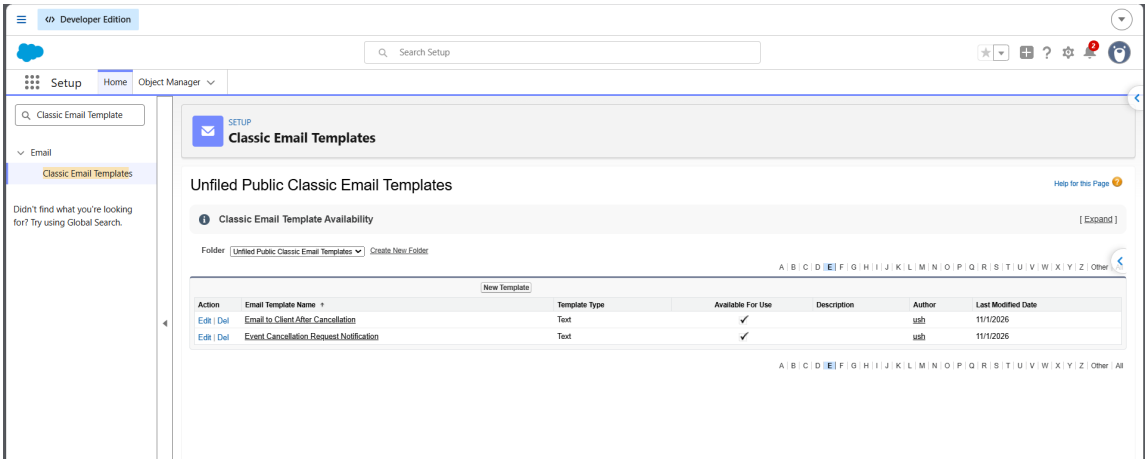


Figure 11: Email templates starting with E

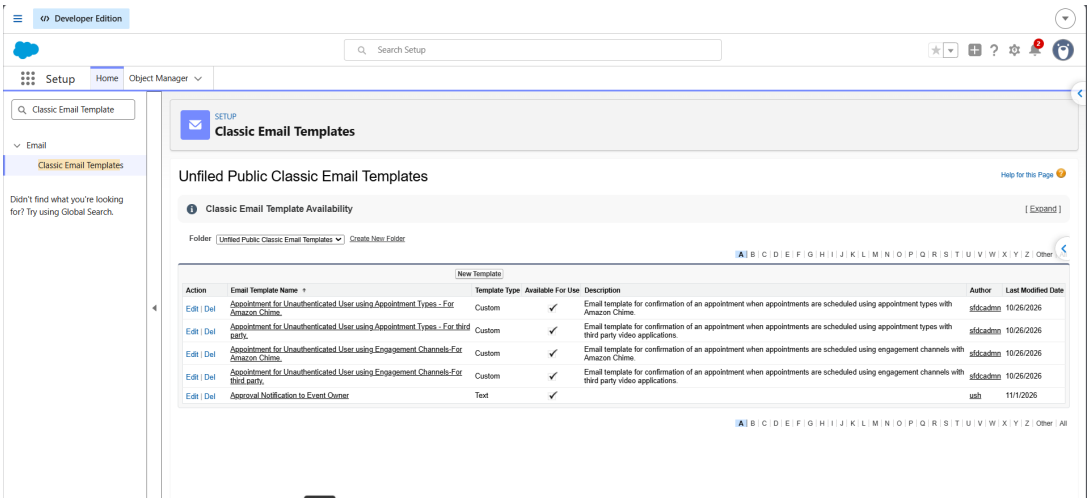


Figure 12: Approval Notification to Event Owner

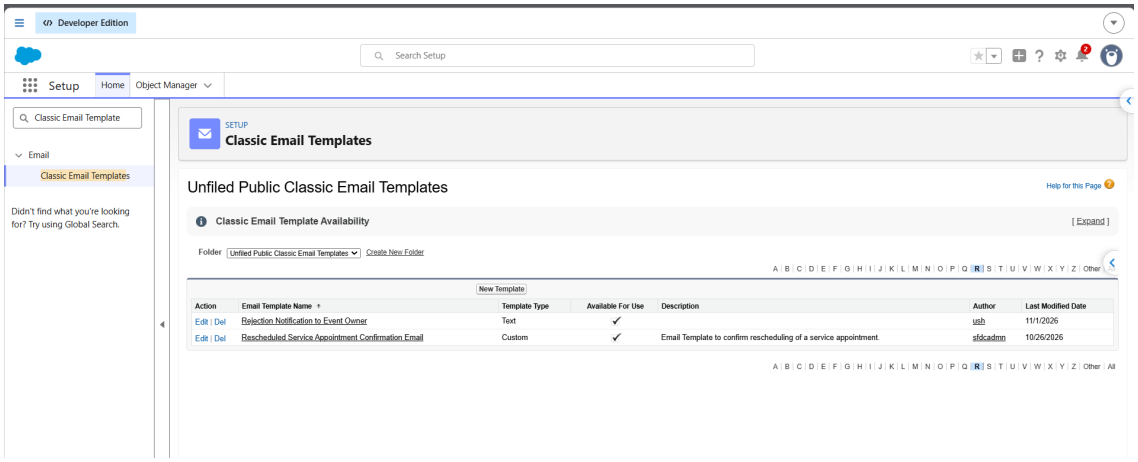


Figure 13: Rejection Notification to Event Owner

7. Approval Process

Part	Configuration
Name	Event_Cancellation_Process on Event
Entry criteria	Event Status equals Pending Cancellation
Initial submitters	Event Owner
Initial submission actions	Record lock and email alert to the Event Coordinator role
Approval step	Manager Review, approver is the Event Admin user
Final approval actions	Event Status set to Canceled and email alert to the event owner
Final rejection actions	Event Status set to Rejected and email alert to the event owner

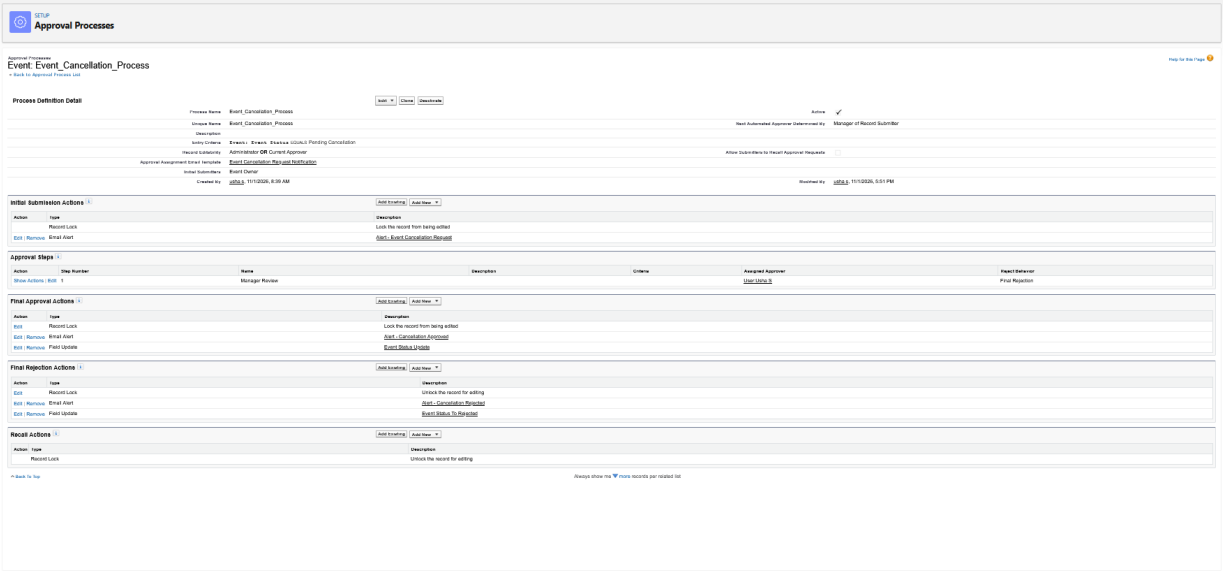


Figure 14: Active approval process with all actions

8. Flow: Client Reminder - 3 Days Before

A record-triggered flow runs on Event when the status is Confirmed. A scheduled path named 3_Days_Before waits until three days before the Event Date and then sends an email to the client with the event name, date, venue and type. The event owner receives a copy. The message subject and body are stored as text template resources.

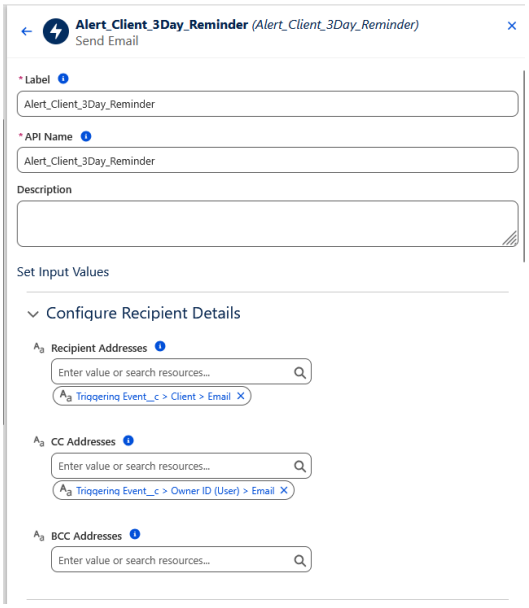


Figure 15: Send Email action (recipient is the client, copy to the owner)

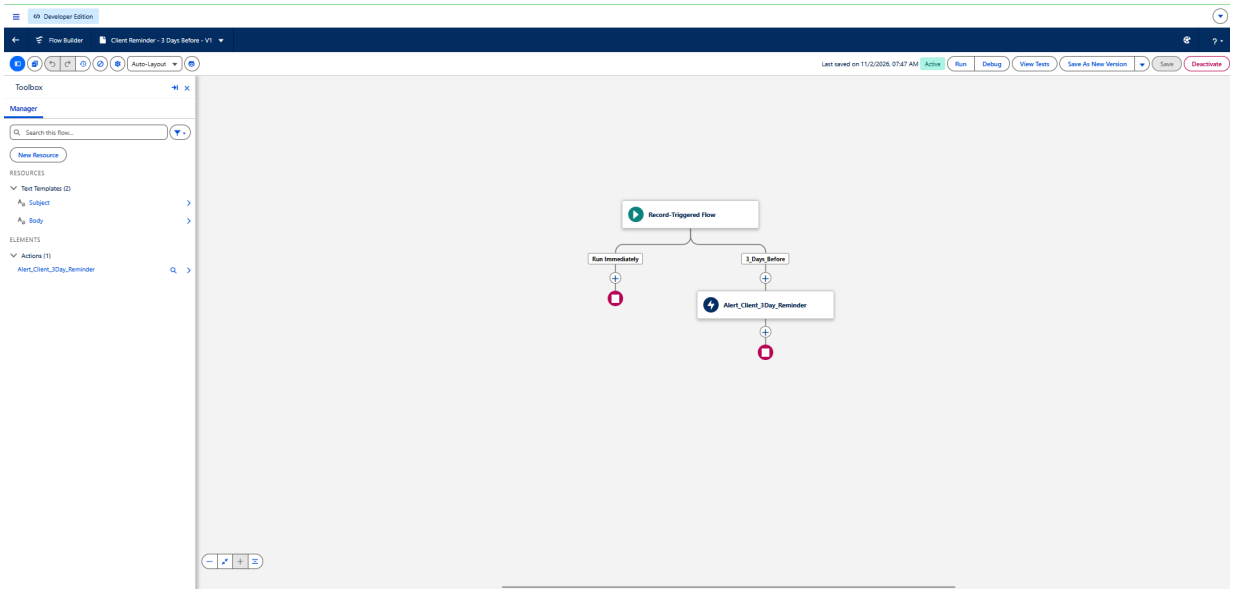


Figure 16: Flow in Flow Builder (active)

9. Apex code

9.1 PreventDoubleBooking (before insert and before update on Event)

The trigger collects the venues of the incoming events, loads the existing events for those venues in a single query, and adds an error when another event is booked at the same venue on the same date. This is bulk safe because the query is outside the loop.

```
trigger PreventDoubleBooking on Event__c (before insert, before update) {
    Set<Id> venueIds = new Set<Id>();
    for(Event__c ev : Trigger.new){
        if(ev.Venue__c != null) venueIds.add(ev.Venue__c);
    }
    Map<Id, List<Event__c>> venueEventMap = new Map<Id, List<Event__c>>();
    for(Event__c e : [SELECT Id, Venue__c, Event_Date__c FROM Event__c
                      WHERE Venue__c IN :venueIds]){
        if(!venueEventMap.containsKey(e.Venue__c)){
            venueEventMap.put(e.Venue__c, new List<Event__c>());
        }
        venueEventMap.get(e.Venue__c).add(e);
    }
    for(Event__c ev : Trigger.new){
        if(ev.Venue__c != null && venueEventMap.containsKey(ev.Venue__c)){
            for(Event__c existing : venueEventMap.get(ev.Venue__c)){
                if(existing.Event_Date__c == ev.Event_Date__c && existing.Id != ev.Id){
                    ev.addError('This Venue is already booked on this date.');
                }
            }
        }
    }
}
```

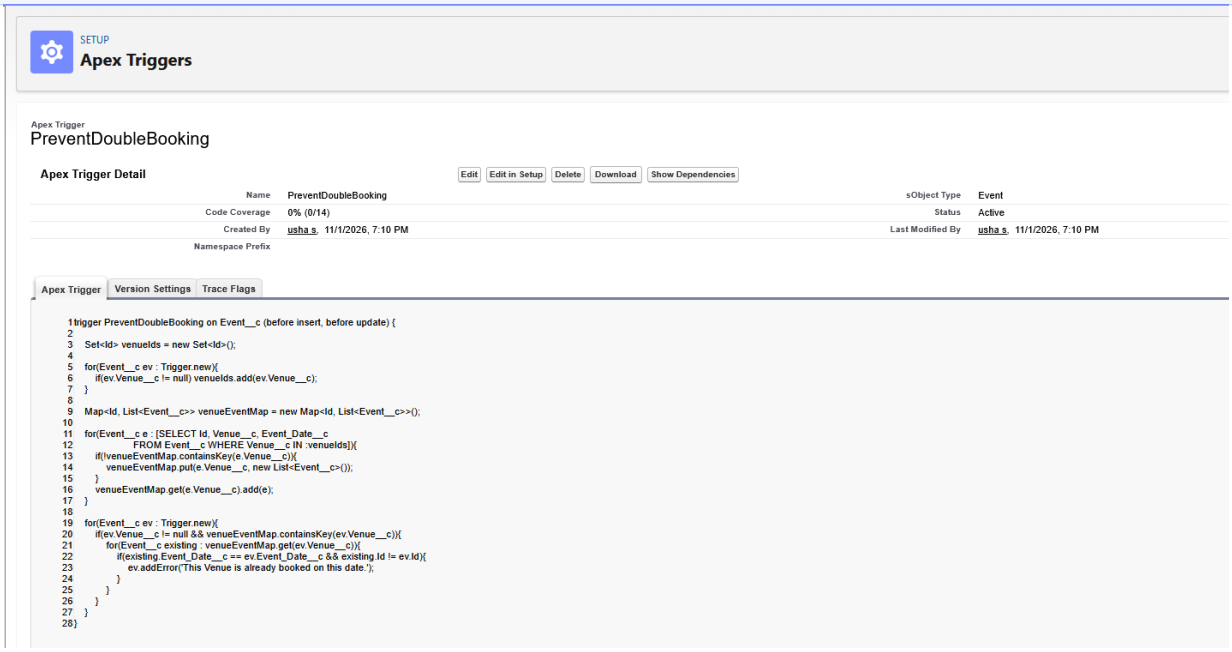


Figure 17: PreventDoubleBooking trigger in Salesforce

9.2 EventTrigger13 and VenueStatusHelper (after insert and after update on Event)

The trigger calls the helper class. A Confirmed event sets its venue to Reserved. A Canceled event sets its venue back to Available.

```
trigger EventTrigger13 on Event__c (after insert, after update) {
    VenueStatusHelper.updateVenueStatus(Trigger.new);
}

public with sharing class VenueStatusHelper {
    public static void updateVenueStatus(List<Event__c> events){
        List<Venue__c> venues = new List<Venue__c>();
        for(Event__c ev : events){
            if(ev.Venue__c != null){
                if(ev.Event_Status__c == 'Confirmed'){
                    venues.add(new Venue__c(Id=ev.Venue__c, Availability_Status__c='Reserved'));
                } else if(ev.Event_Status__c == 'Canceled'){
                    venues.add(new Venue__c(Id=ev.Venue__c, Availability_Status__c='Available'));
                }
            }
        }
        if(!venues.isEmpty()) update venues;
    }
}
```

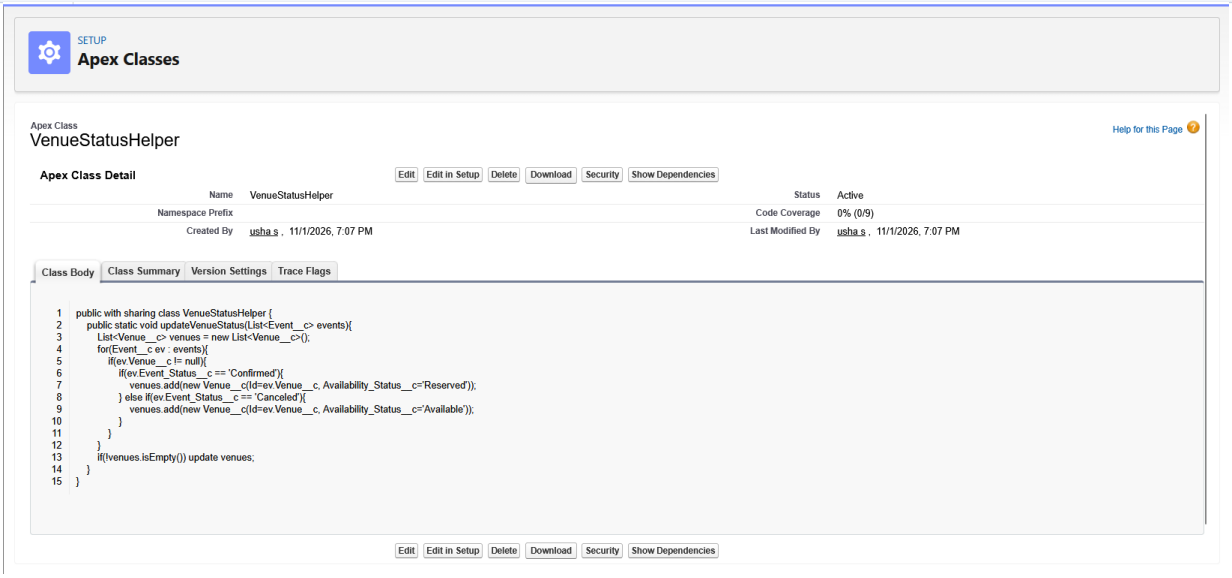


Figure 18: VenueStatusHelper class

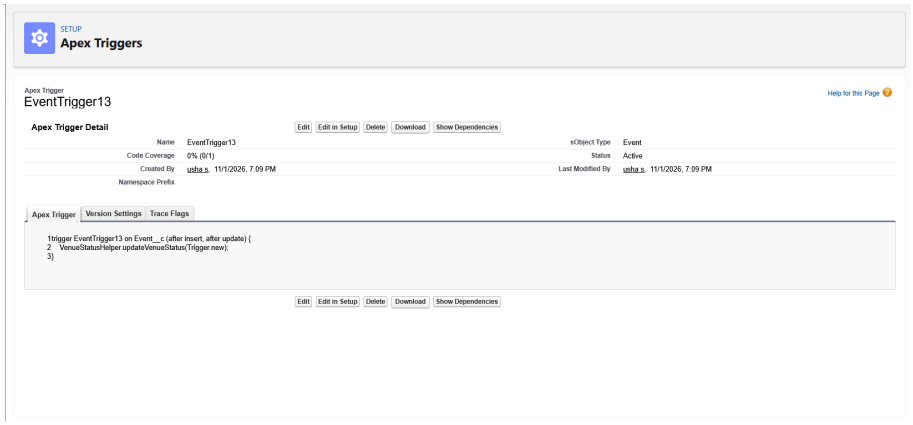


Figure 19: EventTrigger13 trigger

9.3 Batch Apex and Scheduled Apex

BatchCompleteEvents selects every event whose date is before today and whose status is not Completed, and sets the status to Completed. ScheduleCompleteEvents starts the batch. The job Daily Event Completion is scheduled to run every day at 8 PM.

```
global class BatchCompleteEvents implements Database.Batchable<sObject> {
    global Database.QueryLocator start(Database.BatchableContext bc){
        return Database.getQueryLocator('SELECT Id, Event_Date__c, Event_Status__c FROM Event__c ' +
            'WHERE Event_Date__c < TODAY AND Event_Status__c != \'Completed\'');
    }
    global void execute(Database.BatchableContext bc, List<Event__c> scope){
        for(Event__c ev : scope){ ev.Event_Status__c = 'Completed'; }
        update scope;
    }
    global void finish(Database.BatchableContext bc){ System.debug('Past events marked as Completed'); }
}

global class ScheduleCompleteEvents implements Schedulable {
    global void execute(SchedulableContext sc){ Database.executeBatch(new BatchCompleteEvents(), 200); }
}
```

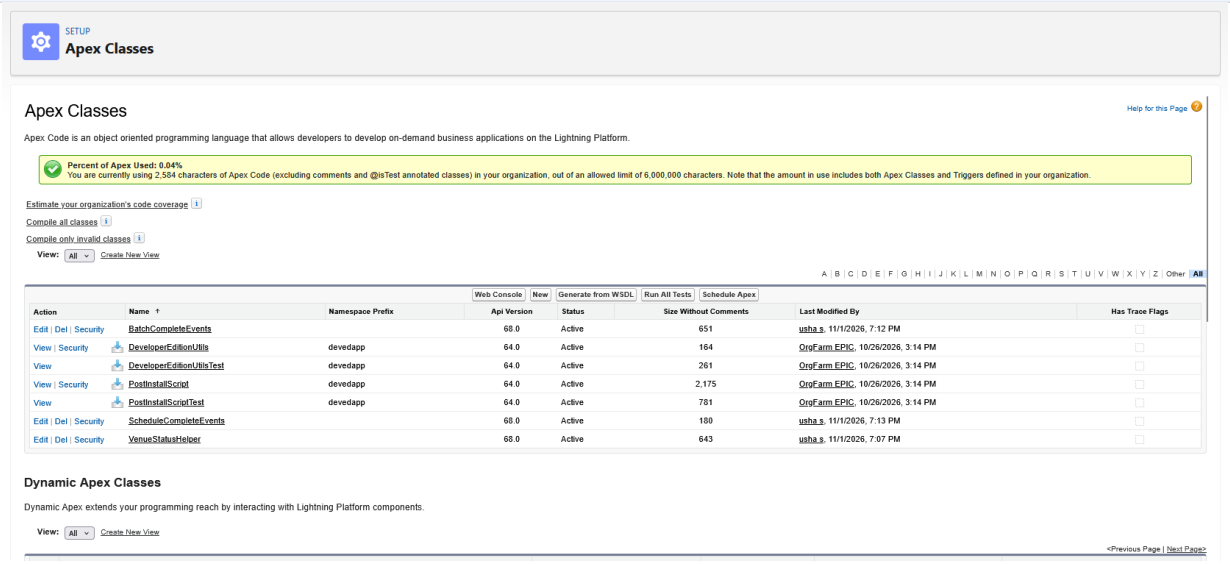


Figure 20: Apex classes (batch, schedulable and helper)

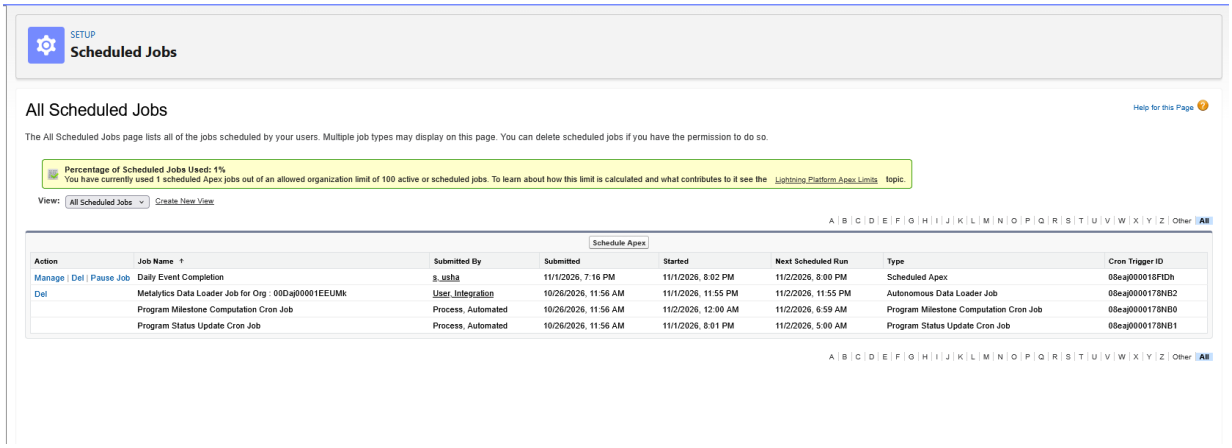


Figure 21: Scheduled job Daily Event Completion